

Examen Final de GRAU-IA

(20 de junio de 2023)

Duración: 2 horas 30 minutos

1. (5 puntos) Dado el preocupante aumento de los niveles de contaminación en las grandes ciudades europeas se quiere incentivar la compra de vehículos híbridos y eléctricos. Para ello nos piden diseñar un programa que, a partir del tipo de conductor, le recomiende uno o varios vehículos de emisiones reducidas del mercado. En esta primera fase el programa solo recomendará vehículos para particulares y autónomos, dejando los coches de empresa y las flotas para futuras versiones de la herramienta.

Al usuario (el conductor habitual del vehículo) se le pregunta por el nombre, la edad, el código postal en el que reside (para saber, por ejemplo, si vive en una área urbana especialmente contaminada) y el presupuesto total (cantidad máxima en euros que se quiere gastar en el vehículo). Y se le pide rellenar toda una serie de datos para conocer sus hábitos de desplazamiento en un año. De cada tipo de desplazamiento que suele realizar se le pregunta si es por ocio o por trabajo, los otros ocupantes del vehículo (de cada uno pregunta nombre y edad), la cantidad de equipaje que suele llevar (en litros), cuantos kilómetros hace al día en ese tipo de desplazamiento, el número de días al mes que se realiza, y el lugar de origen y destino de ese desplazamiento. De cada lugar de desplazamiento se quiere saber su tipología (urbano, interurbano asfaltado, tierra, abrupto) y si dispone de una estación de carga para vehículos eléctricos.

Sobre los vehículos que queremos recomendar (coches y motos) tenemos información almacenada sobre la marca, el modelo, el número máximo de plazas (entre 2 y 7), el tipo de motorización (eléctrico, híbrido o híbrido enchufable), la potencia fiscal (en caballos), el tiempo de carga total de las baterías (en minutos, 0 para los híbridos), el consumo homologado (en litros/100 km), la autonomía homologada (en km), las emisiones homologadas de CO₂ (en gr/km) y el precio base (en euros). En el caso de las motos tenemos también el tipo de moto (scooter, triciclo, motocicleta) y su velocidad máxima (en km/h). En el caso de los coches solo realizará recomendaciones en 6 categorías: micro-coche, turismo, monovolumen, todocamino-compacto, todocamino o todoterreno. En el caso de los micro-coches, turismos, monovolumenes y todocaminos-compactos tendremos también la longitud del coche (en metros, con decimales), al ser un dato relevante en la compra. En los micro-coches tendremos también el ancho del vehículo (en metros, con decimales), y en turismos y todocaminos-compactos sabemos si se abaten los asientos traseros o no. Otra información relevante disponible para todocaminos-compactos, todocaminos y todoterrenos es el tipo de tracción (4x2, 4x4 o 4x4 desconectable), la altura libre de la carrocería al suelo (en cm) y si dispone de asistente para pendientes. Los compradores de todoterrenos son un público muy exigente, y por ello en este caso tenemos además información sobre si la caja de cambios dispone de reductora, la profundidad de vadeo (en cm), el ángulo de ataque, ángulo ventral y ángulo de salida (los tres en grados).

Los expertos en movilidad nos han indicado una serie de criterios importantes a la hora de escoger un vehículo:

- El **tipo de presupuesto** disponible, que se clasifica en bajo (por debajo de los 10000 euros), medio (entre 10000 y 25000 euros) y elevado (por encima de 25000 euros);
- El **número de pasajeros** medio de los desplazamientos, que se clasifica en tres grupos: de 1 a 2 pasajeros, de 3 a 5 pasajeros y de 6 a 7 pasajeros;
- La **necesidad de equipaje** media, que se clasifica en cuatro grupos: muy baja (por debajo de los 10 litros), baja (entre 10 y 100 litros en la mayoría de desplazamientos), media (entre 100 y 250 litros de equipaje medio, o si se lleva un menor de 10 años abordo), y elevada (más de 250 litros de equipaje, o si se lleva dos o más menores de 10 años abordo).
- El **trayecto medio**, que se clasifica en corto (una mayoría de desplazamientos por debajo de 50 km), medio (desplazamientos con una media entre 50 y 200 km) y largo (con una media de más de 200 km por desplazamiento)
- Los **kilómetros al año**, en que se distinguen tres grupos: por debajo de 2000 km/año, entre 2000 y 10000 km/año, y por encima de los 10000 km/año.
- El tipo de **terreno** en el que se suele conducir principalmente, distinguiendo entre cuatro grupos: conductores mayormente urbanos, o mayormente interurbanos, o mixtos (conductores que ocasionalmente salen de vías asfaltadas) o bien off-road (conductores que conducen más del 25 % de sus kilómetros en tierra o que conducen en algún momento por terreno abrupto);
- La **disponibilidad de cargadores**, que estima la facilidad para encontrar una estación de carga en los lugares de origen y destino de los desplazamientos. Se considera una disponibilidad alta si

al menos el 90 % de los lugares de origen y destino de los desplazamientos de trabajo y el 75 % de los lugares en desplazamientos de ocio disponen de estaciones de carga, disponibilidad media si los porcentajes son del 75 % y 50 % respectivamente, y disponibilidad baja en caso contrario.

Los expertos nos han dado también ejemplos de otros criterios para seleccionar los vehículos a recomendar:

- Podemos determinar que la prioridad principal del usuario es un consumo reducido si sus trayectos suelen ser cortos o medios y si suele hacer más de 10000 kilómetros al año. Pero su prioridad será la autonomía si sus trayectos son medios o largos y si suele hacer menos de 10000 km/año.
- Para escoger el tipo de motorización más adecuada para el usuario se combinan varios factores. Los vehículos eléctricos suelen ser más adecuados cuando la principal prioridad es el consumo, si el trayecto medio es corto y si hay una disponibilidad de cargadores media o elevada. Los vehículos híbridos suelen ser más adecuados cuando la principal prioridad es la autonomía, si el trayecto medio es largo y si la disponibilidad de cargadores es baja. Los vehículos híbridos enchufables combinan lo mejor de los vehículos eléctricos y los híbridos, son aptos para usuarios cuya prioridad sea la autonomía o el consumo y si la disponibilidad de cargadores es media, pero no son aptos para presupuestos bajos.
- Para escoger el tipo de vehículo adecuado para el usuario se tienen en cuenta otros factores. Las motos son recomendables para tipos de presupuesto bajos, cuando el terreno no es off-road, se llevan de 1 a 2 pasajeros y la necesidad de equipaje es muy baja. Los micro-coches son más adecuados para presupuestos bajos o medios, cuando el terreno es mayormente urbano, se llevan 5 o menos pasajeros y la necesidad de equipaje es baja. Los turismos son más adecuados para presupuestos medios, cuando el terreno no es mixto ni off-road, se llevan 5 o menos pasajeros y la necesidad de equipaje es media. Los monovolúmenes son más adecuados para presupuestos altos, cuando el terreno no es mixto ni off-road, se llevan entre 3 y 7 pasajeros y la necesidad de equipaje es elevada. Los todocaminos compactos son más adecuados para presupuestos medios, uso en terrenos mixtos, cuando se llevan 5 o menos pasajeros y la necesidad de equipaje es media. Los todocaminos son más adecuados para presupuestos altos, uso en terrenos mixtos, cuando se llevan entre 3 y 7 pasajeros y la necesidad de equipaje es elevada. Finalmente los todoterrenos son aptos para terrenos off-road.

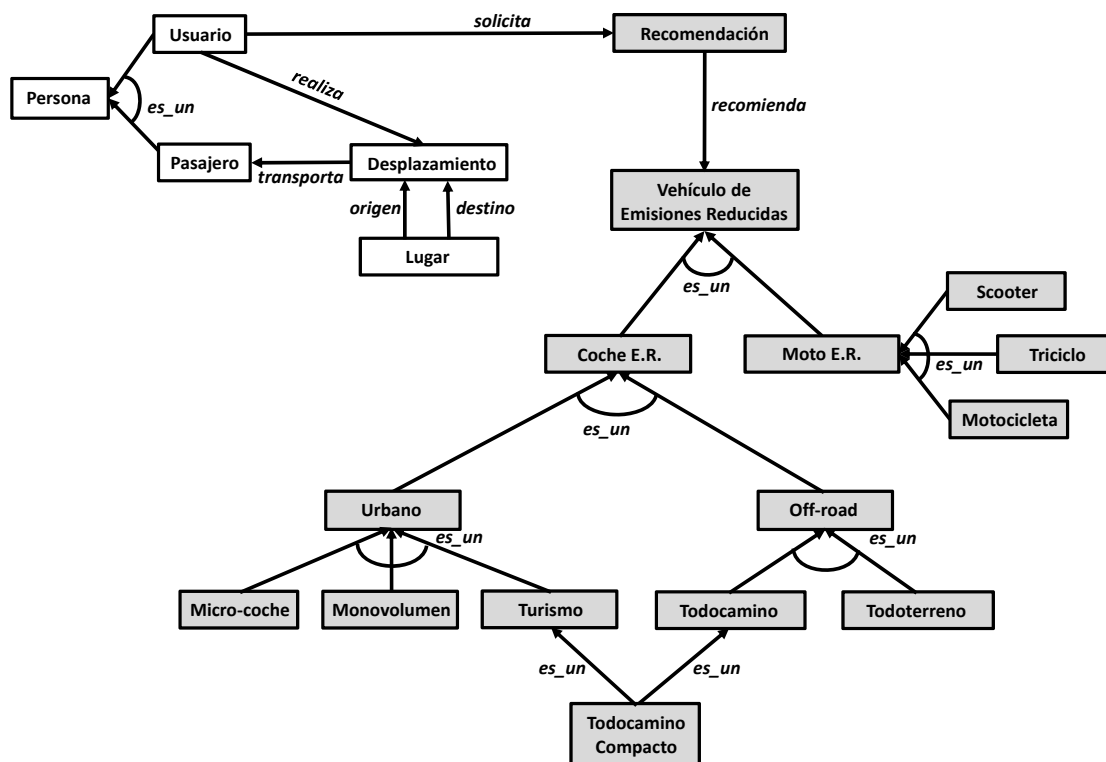
El sistema dará dos tipos de salidas:

- Para aquellos usuarios con la mayoría de trayectos por debajo de los 200 km y que hacen menos de 2000 km al año el sistema intentará concienciarlos de que no necesitan tener un coche en propiedad, y que lo mejor es que combinen el uso del transporte público con el uso esporádico de un vehículo alquilado en una empresa de renting. Eso si, se le aconsejará el tipo de vehículo ecológico (moto, micro-coche, turismo, monovolumen, todocamino-compacto, todocamino o todoterreno) y el tipo de motorización (eléctrico, híbrido o híbrido enchufable) más adecuado a sus necesidades.
 - Para el resto de usuarios recomendará la compra de un vehículo ecológico y mostrará la lista de modelos disponibles en el mercado, incluyendo el precio y los impuestos. La aplicación dispone de una función `mostrar_lista_vehículos(tipo_vehículo, tipo_motorización, presupuesto_total, código_postal)` que dado el tipo de vehículo escogido, el tipo de motorización, el presupuesto total (en euros) y el código postal del usuario muestra por pantalla la lista de vehículos disponibles en el mercado que corresponden al tipo de motorización y vehículo recomendados y cuyo precio base más el importe del impuesto de matriculación que ha de pagar en su zona de residencia están por debajo del presupuesto total.
- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción e identificando los atributos más relevantes. Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. Justifica las decisiones de diseño que has realizado. En el caso de las subclases de una clase, justifica con un texto breve la necesidad de crear dichas subclases. Para cada concepto de la ontología lista todos sus atributos (para cada atributo se ha de indicar su nombre y su tipo, y en el caso de las enumeraciones, sus valores posibles). [Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al apartado siguiente.]

En este problema la ontología ha de incorporar, como mínimo, todos los conceptos que forman parte de la entrada del problema: la información que se obtiene sobre el usuario, sus desplazamientos y las personas que le acompañan. También ha de incorporar los conceptos de la solución (en gris

en la figura a continuación), que incluyen la recomendación a realizar y la taxonomía de vehículos de emisiones reducidas (E.R. en la figura). Se ha penalizado el no decir qué parte de la ontología describe el problema y cuál la solución (se pedía en el enunciado). Se ha penalizado que ambos lados queden desconectados: una ontología desconectada no tiene sentido, y además debería aparecer de forma natural la clase 'recomendación' para almacenar los datos de la solución abstracta.

De la ontología resultante cabe remarcar que existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo tipo. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes y/o relaciones diferentes. Ese es el caso de las subclases de **Persona**, **Urbano** y **Off-Road**. Sobre **Persona**, cabe destacar que debería seguir existiendo la subclase **Pasajero** para mantener el nivel de abstracción. Sobre las otras dos superclases (**Urbano** y **Off-Road**) son claras debido a la cantidad de atributos compartidos, y aunque el nombre no aparece directamente en el enunciado como categoría, se puede deducir por las reglas posteriores. También se han aceptado nombres alternativos. No se siguió del todo el criterio de sólo separar las categorías en función de atributos/relaciones en el caso de las subclases de **Moto E.R.**, que a pesar de que las subclases no tienen ni atributos ni relaciones diferenciadas se han añadido siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía tengan un nivel de granularidad similar (en este caso la rama hermana del **Coche E.R.**). Se ha puntuado parcialmente el dejarlas como atributo tipo siempre y cuando se haya razonado por qué se ha preferido esta opción. Una cosa a remarcar del modelo es que se ha decidido que **Todocamino Compacto** tenga herencia múltiple de **Turismo** y de **Todocamino** ya que de esta manera obtiene todos los atributos que necesita. En caso que necesitáramos usar la ontología con un razonador que no soporte herencia múltiple podríamos hacer que **Todocamino Compacto** herede únicamente de **Todocamino** y añadirle los atributos de **Turismo** que le faltan. Ambas opciones son igual de válidas. Se ha penalizado dejar el **Todocamino Compacto** sin relaciones jerárquicas a **Todocamino**.



Atributos: a continuación se listan los atributos mínimos para representar la información mencio-

nada explícitamente en el enunciado y la necesaria para el apartado b) del problema. Hay otros atributos que se deberían añadir (como el tipo de cambio de marchas, la aceleración de 0 a 100 km/h o los extra que lleva el coche) si quisiéramos una ontología lo más general y reutilizable posible. Es penalizable no listar los atributos que forman parte del problema y solución abstractos (para la segunda parte del problema).

- **Persona:** nombre (string), edad (entero);
- **Usuario:** presupuesto_total (euros), código_postal (entero), media_pasajeros (real), media_niños_pasaje (real), media equipaje (litros), media_trayecto (km), total_kilometros_año (km), media_cargadores_lugar_trabajo (real), media_cargadores_lugar_ocio (real), *Tipo_Presupuesto* (enumeración: {bajo, medio, elevado}), *Num_Pasajeros* (enumeración: {1a2, 3a5, 6a7}), *Necesidad_Equipaje* (enumeración: {muy_baja, baja, media, elevada}), *Terreno* (enumeración: {may_urbano, may_interurbano, mixto, off-road}), *Disponibilidad_Cargadores* (enumeración: {alta, media, baja}), *Trayecto_Medio* (enumeración: {corto, medio, largo}), *Km_Año* (enumeración: {<2000, 2000a10000, >10000}), *Prioridad* (enumeración: {consumo, autonomía, indistinta});
- **Desplazamiento:** motivo (enumeración: {ocio, trabajo }), litros_equipaje (entero), km_dia (entero), dias_mes (entero);
- **Lugar:** tipología (enumeración: {urbano, interurbano asfaltado, tierra, abrupto}), estación_carga? (booleano);
- **Recomendación:** *TIPO_VEHICULO* (enumeración: {moto, micro-coche, turismo, monovolumen, todocamino-compacto, todocamino, todoterreno}), *TIPO_MOTORIZACION* (enumeración: {eléctrico, híbrido, híbrido_enchufable}), *TIPO_ADQUISICION* (enumeración: {compra, renting});
- **Vehículo de Emisiones Reducidas:** marca (string), modelo (string), max_plazas (entero), motorización (enumeración: {eléctrico, híbrido, híbrido_enchufable}), potencia_fiscal_cv (entero), tiempo_carga (minutos), consumo (l/100km), autonomía (km), emisiones_CO2 (gr/km), precio_base (euros);
- **Moto E.R.:** velocidad_máxima (entero);
- **Coche E.R.:** num_puertas (entero), volumen_maletero (litros);
- **Urbano:** longitud (metros);
- **Micro-coche:** ancho (metros);
- **Turismo:** asientos_traseros_abatibles? (booleano);
- **Off-road:** tracción (enumeración: {4x2, 4x4, 4x4_desconectable}), altura_al_suelo (cm), asistente_pendientes? (booleano);
- **Todoterreno:** reductora? (booleano), profundidad_vadeo (cm), ángulo_ataque (grados), ángulo_ventral (grados), ángulo_salida (grados);

Como se puede ver, hemos decidido también representar las características abstractas del problema y las de la solución abstracta (son los que tienen el nombre en *Cursiva* o *CURSIVA*, respectivamente). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

Nótese que de cara a la evaluación se ha valorado especialmente la creación de superclases y subclases donde correspondiera (especialmente la taxonomía de vehículos).

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística. Justifica si el tipo de solución que se pide requiere de refinamiento o no. Lista las variables que forman parte del Problema Abstracto (nombre y valores posibles). Lista las variables que forman parte de la Solución Abstracta (nombre y valores posibles). Da al menos 3 ejemplos de reglas (suficientemente variadas, utilizando diferentes conceptos) para cada una de las fases de esta metodología, usando los conceptos de la ontología desarrollada en el apartado anterior. Para las reglas podéis usar la notación de alto nivel que usamos en los ejercicios de problemas (*si ... entonces* ...) o una notación más próxima a la del lenguaje CLIPS (siempre que queden claro los elementos del antecedente de la regla y del consecuente).

Para resolverlo mediante clasificación heurística debemos identificar en el problema las diferentes fases y elementos de esta metodología. En este caso hay solo una opción posible: la solución que pide el enunciado solo puede ser una solución concreta, ya que no es correcto acceder en el nivel de solución abstracta a valores concretos como el presupuesto total del cliente o el código postal para construir la lista de vehículos recomendados.

Una vez sabemos cuantas fases requiere nuestra solución ya podemos enunciar la solución completa. El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por toda la información detallada sobre el usuario, sus desplazamientos, los lugares por los que pasa y los pasajeros, tal y como los hemos identificado en la ontología. En este caso hace falta un preproceso de los datos, ya que en la mayoría de reglas de abstracción se necesita saber los valores medios (kilómetros recorridos, pasajeros en trayecto, ...) o mayoritarios (tipo de terreno por el que se circula). Esto requiere el cálculo de una serie de valores (por ejemplo, multiplicar para cada tipo de desplazamiento los $\text{km_día} * \text{días_mes} * 12$ para saber los km_año recorridos en ese desplazamiento, y usar ese valor no solo para calcular los km_año totales sino también la proporción de km por cada tipo de terreno por el que el usuario circula, basándose en la tipología de los lugares de origen y destino). Suponemos que tenemos métodos para realizar todos estos cálculos a partir de los datos que introduce el usuario, y estos quedan almacenados en la ontología mediante atributos extra que añadimos para tal efecto: *media_pasajeros*, *media_niños_pasaje*, *media_equipaje*, *media_trayecto*, *total_kilometros_año*, *media_cargadores_lugar_trabajo*, *media_cargadores_lugar_ocio*, etc.

El segundo elemento son los problemas abstractos, estos estarán definidos a partir de las siete características que menciona el enunciado (*Tipo_Presupuesto*, *Num_Pasajeros*, *Necesidad_Equipaje*, *Terreno*, *Disponibilidad_Cargadores*, *Trayecto_Medio*, *Km_Año*) al que añadimos un criterio más, la *Prioridad*, ya que esta variable está más relacionada con la descripción del problema abstracto (las necesidades del usuario) que con la descripción de la solución abstracta (el vehículo más adecuado para el usuario).

Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si Usuario.presupuesto < 10000 euros entonces Usuario.Tipo_Presupuesto=bajo;
- si Usuario.presupuesto > 25000 euros entonces Usuario.Tipo_Presupuesto=elevado;
- si Usuario.media_pasajeros ≤ 2 entonces Usuario.Num_Pasajeros=1a2;
- si Usuario.media_pasajeros > 5 entonces Usuario.Num_Pasajeros=6a7;
- si Usuario.media_equipaje > 25 litros o Usuario.media_niños_pasaje > 1,9 entonces Usuario.Necesidad_Equipaje=elevada;
- si Usuario.media_trayecto < 50km entonces Usuario.Trayecto_Medio=corto;
- si Usuario.total_kilometros_año < 2000 entonces Usuario.Km_Año='<2000';

El tercer elemento son las soluciones abstractas. En este caso tenemos tres aspectos diferentes que analizar (el tipo de vehículo, el tipo de motorización y el tipo de adquisición) y hemos creado una variable cualitativa para cada aspecto, que corresponde con los atributos del concepto **Recomendación** del apartado anterior: *TIPO_VEHICULO* (enumeración: {moto, micro-coche, turismo, monovolumen, todocamino-compacto, todocamino, todoterreno}), *TIPO_MOTORIZACION* (enumeración: {eléctrico, híbrido, híbrido_enchufable}), *TIPO_ADQUISICION* (enumeración: {compra, renting}).

Para calcular el valor de la propiedad abstracta *Prioridad* a partir de las otras variables abstractas del problema se requieren una serie de reglas de inferencia previas a la asociación heurística (aunque se han dado como buenas si están categorizadas dentro de la asociación heurística ya que trabajan sobre el lenguaje de problema abstracto), como por ejemplo:

- si Usuario.Trayecto_Medio=(corto o medio) y Usuario.Km_Año='>10000' entonces Usuario.Prioridad=consumo;
- si Usuario.Trayecto_Medio=(medio o largo) y Usuario.Km_Año=('<2000' o '2000a10000')
- si Usuario.Trayecto_Medio= largo y Usuario.Km_Año='>10000' entonces Usuario.Prioridad=indiferente;
- si Usuario.Trayecto_Medio=corto y Usuario.Km_Año=('<2000' o '2000a10000') entonces Usuario.Prioridad=indiferente;

Para ligar los problemas abstractos con las soluciones abstractas necesitaremos reglas de asociación heurística, como por ejemplo:

- si Usuario.Prioridad=consumo y Usuario.Trayecto_Medio=corto y Usuario.Disponibilidad_Cargadores=(media o elevada) entonces Recomendación.TIPO_MOTORIZACION=eléctrico;
- si Usuario.Prioridad=autonomía y Usuario.Trayecto_Medio=largo y Usuario.Disponibilidad_Cargadores=baja entonces Recomendación.TIPO_MOTORIZACION=híbrido;
- si Usuario.Tipo_Presupuesto=bajo y Usuario.Num_Pasajeros='1a2' y Usuario.Terreno≠off-road y Usuario.Necesidad_Equipaje=muy_baja entonces Recomendación.TIPO_VEHÍCULO=moto;
- si Usuario.Tipo_Presupuesto=(bajo o medio) y Usuario.Num_Pasajeros=('1a2' o '3a5') y Usuario.Terreno=urbano y Usuario.Necesidad_Equipaje=baja entonces Recomendación.TIPO_VEHÍCULO=micro-coche;
- si Usuario.Terreno=off-road entonces Recomendación.TIPO_VEHÍCULO=todoterreno;
- si Usuario.Trayecto_Medio=(corto o medio) y Usuario.Km_Año='<2000' entonces Recomendación.TIPO_ADQUISICIÓN=renting;

El cuarto y último elemento son las soluciones concretas. En este caso corresponde a la generación de las dos posibles salidas del sistema, aprovechando las soluciones abstractas y refinándolas con datos concretos del problema para poder generar las salidas descritas en el enunciado. En este caso necesitamos solo dos reglas de producción para el refinamiento. Suponemos la existencia de un método **recomendar_renting** para generar la pantalla en caso de que se le quiera recomendar al usuario la opción del renting de un vehículo.

- si Recomendación.TIPO_ADQUISICIÓN=renting entonces recomendar_renting(Recomendación.TIPO_VEHÍCULO, Recomendación.TIPO_MOTORIZACION);
- si Recomendación.TIPO_ADQUISICIÓN=compra entonces mostrar_lista_vehículos(Recomendación.TIPO_VEHÍCULO, Recomendación.TIPO_MOTORIZACION, Usuario.presupuesto_total, Usuario.código_postal)

2. (5 puntos) El mago Rincewind, tras retirarse de sus aventuras, ha creado un gremio inteligente de fabricación de regentes y artilugios mágicos. Este gremio es un centro generalista, encargado de la generación de prototipos a partir de pasos, organizando su producción entre diversas máquinas y montando planes de producción. Sin embargo, el encontrar estos planes a mano es bastante tedioso, así que ha pedido a sus becarios un sistema que se encargue de planificar la producción.

Cuando un problema llega al gremio, el objetivo es la secuencia de pasos y máquinas que se deben usar para fabricarlo. Para cada componente o artilugio sabemos qué máquina puede usarse para fabricarlo y qué productos necesita, que dependen de cada problema específico. Un componente o artilugio puede ser fabricado de muchas formas diferentes (ej. se puede conseguir sustrato con centrifugadora:suero+bilis de basilisco, o con materializador:arcana o tienda:-), pero podéis asumir que no hay dos formas diferentes de hacerlo con la misma máquina. Cada componente requiere siempre de una máquina, pero puede requerir de un número de ingredientes arbitrario. Por falta de almacenamiento, podéis asumir que nunca habrá más de un ingrediente del mismo tipo (ej. no puede haber dos sueros a la vez).

El problema tiene una complejidad adicional: la fabricación tiene una duración. Esto quiere decir que meter los productos en la máquina no implica tener el resultado inmediatamente. Asumimos que todas las máquinas tardan exactamente el mismo tiempo en generar cualquier componente, y que el tiempo de cargar las máquinas de ingredientes es despreciable. Sin embargo, una máquina no puede hacer dos componentes a la vez. Queremos minimizar el tiempo que tardamos en tener todos los productos pedidos en el problema al gremio.

- a) (4 puntos) Describe el dominio (incluyendo predicados, acciones, etc.) usando PDDL. La puntuación

será proporcional a cuánto del problema se ha podido modelar (eg. poder generar productos < generar productos con un número de ingredientes variable < que las máquinas tarden en producir < minimizar tiempo de producción).

Errores comunes vistos durante las propuestas

- Para hablar de 'recetas', mucha gente ha utilizado (ingrediente ?c ?i). Esto sería incorrecto porque cuando un ingrediente tiene diversas posibles recetas (eg. Molde) se pedirá todos los ingredientes de todas las recetas.
- Escribir un forall como primera parte de la precondition (extremadamente ineficiente ya que es lo primero que se comprueba).
- Contabilizar el tiempo (en paralelo) como el máximo número de veces que se utiliza una máquina (o variantes de esto). Por ejemplo, si el plan completo es: Máq.1 produce algo que va a Máq.2, esto se contabilizaría como tiempo 1, pero no es realmente paralelizable.
- No comprobar que existe una receta que usa la máquina (eg. permitiendo hacer 'fabricar' de patito en materializador, matcheando en el forall ningún 'requiere' porque no se puede hacer).

Para el problema, nosotros introducimos el componente del método (que es un identificador de qué proceso/receta se está usando). No es necesario dado la frase del enunciado 'podéis asumir que no hay dos formas diferentes de hacerlo con la misma máquina'. Para ello, habría que reescribir los predicados (metodo_fabrica ?c -componente ?ma -maquina ; metodo_requiere ?c -componente ?ma -maquina ?i -componente) y prescindiríamos de metodo_usa.

```
(define (domain RincewindFactory)

  (:requirements :strips :adl :typing :fluents )

  (:types componente maquina metodo - object)

  (:predicates
    (tengo ?c - componente)
    (en_fabricacion ?c - componente)
    (en_uso ?m - maquina)
    (metodo_usa ?me - metodo ?ma - maquina)
    (metodo_fabrica ?me - metodo ?c - componente)
    (metodo_requiere ?me - metodo ?c - componente)
  )

  (:functions
    (tiempo)
  )

  (:action esperar
    :parameters ()
    :precondition ()
    :effect ( and
      (forall (?m - maquina) (not (en_uso ?m)))
      (forall (?c - componente) (and
        (when (en_fabricacion ?c)
          (not (en_fabricacion ?c))
          (tengo ?c)
        )
      )
      (increase (tiempo) 1 )
    )
  )
)
```

```

(:action poner_a_fabricar
:parameters (?ma -maquina ?me -metodo ?a_fabricar - componente)
:precondition (and
  (not (en_uso ?ma)) (metodo_usa ?me ?ma) (metodo_crea ?me ?a_fabricar)
  (forall (?c - componente) (when (metodo_requiere ?me ?c) (tengo ?c)))
)
:effect (and
  (en_uso ?ma) (en_fabricacion ?a_fabricar)
  (forall (?c - componente)
    (when (metodo_requiere ?me ?c)
      (not (tengo ?c))
    )
  ))
) ) )

```

- b) (1 puntos) El gremio ha recibido una petición de hacer un talismán: ‘patito de goma aprueba-exámenes’, seguramente de uno de los becarios. El gremio dispone ahora mismo de madera y plástico. Debajo se listan los procesos de fabricación posibles. Repersenta el problema en PDDL acorde con vuestra propuesta de dominio del apartado anterior.

Fabricado	Máquina	Ingredientes
Patito	M.Ensamblaje	Pico, cuerpo, ojos
Cuerpo	Soplador	Plástico, molde
Plástico	Tienda	-
Plástico	Materializador	-
Molde	Fundición	Plástico
Molde	CNC	Madera
Pico	Simbolificador	Pico real
Pico real	Descuartizapatos2000	Pato
Ojos	Simbolificador	Espíritu, plástico
Espíritu	Materializador	-
Pato	Materializador	-
Pato	Tienda	-

```

(define (problem patito)
  (:domain RincewindFactory)
  (:objects
    PATITO PICO CUERPO OJOS ... - componente
    ENSAMBLAJE SOPLADOR TIENDA ... - maquina
    M1 ... M12 - metodo ; Tantos como lineas en la tabla
  )
  (:init
    (= (tiempo) 0)
    (tengo MADERA) (tengo PLASTICO)
    (metodo_usa M1 ENSAMBLAJE) (metodo_fabrica M1 PATITO)
    (metodo_requiere M1 PICO)
    (metodo_requiere M1 CUERPO)
    (metodo_requiere M1 OJOS)
    ...
    (metodo_usa M3 TIENDA) (metodo_fabrica M3 PLASTICO)
    ...
  )
  (:goal (tengo PATITO))
  (:metric minimize (tiempo))
)

```

)

Las notas se publicaran el día **28 de junio**.

GRAU-IA Final Exam

(June 20, 2023)

Time: 2 hours 30 minutes

1. (5 points) Due to worrying increase in pollution levels across all big European cities, there is the will to promote the acquisition of electric and hybrid vehicles. Therefore we receive the command to design a program that, using information about the driver, is capable of recommending one or several low-emission vehicles in the market. In this first version the program will only recommend vehicles to individuals, leaving recommendations for companies and fleets for future releases.

The user (that is, the usual driver) will be questioned about his/her name, age, the zip post code where he/she lives (to infer, for instance, if he/she lives in a highly polluted urban area) and the total budget (maximum amount, in euro, that the user is willing to spend on the vehicle). The user should also introduce some data about his/her travel habits along the year. For each travel type that he/she uses to do the user should state if the main motivation for the travel is due to work or leisure, the other passengers in the car (for each the tool asks for the name and age), the amount of luggage that is usually carried in (in liters), how many kilometers per day are made for that travel type, the number of days per month that travel type is usually driven, and origin and destination locations. For each of those locations we also want to know their type (city, asphalt road, dirt road, rocky) and whether there is any electric vehicle charging station available.

For each of the vehicles to be recommended (cars and motorcycles) we have collected data about the make, the model, the maximum number of occupants (from 2 to 7), the engine type (electric, hybrid or plug-in hybrid), the tax horsepower (in cv), the time to battery full-charge (in minutes, 0 for hybrids), the fuel economy (in liters per 100 km), the maximum range (in km), the CO₂ emissions (in gr/km) and car price (in euro). In the case of motorcycles we also know the type (scooter, motorbike, trike) and its top speed (in km/h). In the case of cars the program will only make recommendations for 6 categories: microcar, compact, multi-purpose vehicle (MPV), compact crossover, crossover and off-road. In the case of microcars, compacts, MPVs and compact crossovers we also know the car length (in meters, with decimals), as this is relevant for their users. In the case of microcars we have also the car width (in meters, with decimals), and for compacts and compact crossovers we also know if the rear seats can be folded down or not. Other relevant information for compact crossovers, crossovers and off-roads is the drivetrain type (4x2, 4x4 or disconnectable 4x4), ground clearance (in cm) and if it has an electronic assistant for hill start/descent. Off-road vehicle buyers are even more demanding on data specs and for those vehicles we also store information about: whether they have a reduction gearbox, the air intake height (in cm), the approach angle, breakover angle and departure angle (all three in grades).

Mobility experts have told us that there exists some relevant criteria to guide the selection of vehicles for users:

- The available **budget type**, which is classified into low (under 10000 euro), medium (from 10000 to 25000 euro) and high (above 25000 euro);
- The average **passenger number** in travels, which is classified into three groups: from 1 to 2 passengers, from 3 to 5 passengers and from 6 to 7 passengers;
- The average **luggage need**, which is classified into four groups: very low (below 10 liters of luggage), low (from 10 to 100 liters in most travels), medium (from 101 to 250 liters, or if there is a passenger 10 years-old or less on-board), and high (more than 250 liters, or if there are two or more passengers on-board which are 10-year-old or less).
- The **average trip**, which is classified into three groups: short (a majority of travels under 50 km), medium (travels with an average between 50 and 200 km) and long (with averages above 200 km per travel)
- The **kilometers per year**, which is classified into three groups: below 2000 km/year, from 2000 to 10000 km/year, and above 10000 km/year.
- The **terrain type** that is mostly driven on, which is classified into four groups: mainly city, mainly asphalt road, mixed (drivers that go to unpaved roads eventually) or off-road (users that drive more than 25 % of their kilometers on dirt roads or that eventually drive on rocky terrain);
- The **charging station availability**, an estimation on how easy is to find a battery charging station in the origin and destination locations of the user's travels. We will consider a high availability if at least the 90 % of the origin and destination locations for work travels and at least 75 % of locations

for leisure travels have battery charging stations available; medium availability if those percentages are 75 % and 50 % respectively, and low availability otherwise.

Mobility experts have also provided us with examples of other criteria they use in their vehicle recommendations:

- We can determine whether the main user priority is fuel economy if his/her average trips use to be short or medium and if he/she uses to drive more than 10000 kilometers per year. But maximum range will be the main priority in case of a user with medium or large average trip and less than 10000 kilometers driven per year.
- To choose the recommended engine type several factors are combined. Electric vehicles are better for those cases where fuel economy is the main priority, with low or medium average trips and a high or medium charging station availability. Hybrid vehicles are better for those cases where maximum range is the main priority, with long average trips and a low charging station availability. Plug-in hybrid vehicles combine the best of electric and hybrid vehicles, they are a good choice for users with any preference on fuel economy or maximum range and if charging station availability is medium, but they are not a good option in case of low budget type.
- To choose the recommended vehicle type other factors are combined. Motorcycles are suitable for low budget types, when terrain type is not off-road, passenger number is from 1 to 2 and there is a very low luggage need. Microcars are better suited for low or medium budget types, when terrain type is mainly city, passenger number is 5 or less and there is a low luggage need. Compacts are better suited for medium budget types, when terrain type is neither mixed not off-road, passenger number is 5 or less and there is a medium luggage need. MPVs are a good choice for high budget types, when terrain type is neither mixed not off-road, passenger number is between 3 and 7 and there is a high luggage need. Compact crossovers are better suited for medium budget types, when terrain type is mixed, passenger number is 5 or less and there is a medium luggage need. Crossovers are the proper choice for high budget types, when terrain type is mixed, passenger number is between 3 and 7 and there is a high luggage need. Finally, off-roads are adequate for users driving on off-road terrain type.

The system will provide two possible outputs:

- For those users with a majority of travels under 200 km and less than 2000 km/year the system will try to convince them that they don't need to buy a car for their private use, and that the best option for them is to combine the use of public transportation with an occasional use of a rent car through a car renting service. The system will also recommend the type of low-emissions vehicle (motorcycle, microcar, compact, MPV, compact crossover, crossover or off-road) they should rent and the engine type to select (electric, hybrid or plug-in hybrid) which are better suited for their mobility needs.
 - For the rest of users the system will recommend to buy a low-emissions vehicle and it will show a list of the available models in the market, including their price and the taxes to be paid. The program has an already programmed method (`show_vehicle_list(vehicle_type, engine_type, total_budget, zip_code)`) that, given the selected vehicle type, the recommended engine type, the user's total budget (in euro) and the zip post code, it shows in screen the list of those vehicles in the market which match the recommended vehicle type and engine type and whose car price plus local taxes (computed thanks to the zip code) is less or equal than the user's total budget.
- a) Define the domain ontology, including all the concepts that appear in this description and properly identifying the most relevant attributes and relations. List which concepts belong to the input data and which concepts belong to the system solution. (Please note: take into account that the ontology may need some modifications to adapt it to next section).
- b) The problem we have described is an analysis problem. Explain how you would solve it by means of heuristic classification, using the concepts, attributes and relations from the ontology you defined in the previous section. Please give a minimum of 3 examples of rules for each of the phases in the methodology.
2. (5 points) Rincewind the wizard, after his retirement from adventurous lifestyle, has set up a guild of intelligent manufacturing of magical reagents and artefacts. This guild intends to be generalistic, and in charge of prototype generation from steps, organising its production between several machines and setting up production plans. However, finding those plans by hand is tedious, so he's put his interns to work on a system for planning production.

Whenever a new task arrives at the guild, the objective of the guild is to list the sequences of steps and machines that must be performed to fulfill the task. For each component or artefact we know which machine can be used to manufacture it and which products it requires, which are task-dependant. A component or artefact can be made in different ways (eg. substrate can be obtained either with a centrifuge:serum+basiliskbilis or with a materialiser:arcana or via store:-), but it can be safely assumed the same product cannot be obtained with the same machine but different ingredients. Each component always requires of a single machine, but it may require an arbitrary number of ingredients. Due to lack of storage space, you can assume that there can't be more than one ingredient of the same type (eg. can't have two serums at once).

The problem holds an additional complexity: fabrication is durative. This means that inputting the ingredients in a machine does not immediately give the result. We assume that all machines take the same time in generating any component, and that the 'loading' time of inputting ingredients in a machine is negligible. However, a machine cannot make two components at once. We want to minimize the time we take to have all products asked for in the task.

- a) (4 points) Describe the domain (including predicates, actions, etc.) using PDDL. The score obtained will be proportional to the amount of the task description is covered (eg. being able to obtain components < generating products with variable number of ingredients < durative manufacturing < minimising production time).
- b) (1 point) The guild has received a petition for a talisman: "Rubber ducky of exam-passing", probably from the interns. The guild currently has 'wood' and 'plastic'. Find listed below the production steps that are possible. Represent the problem in PDDL in accordance with your domain proposal.

Result	Machine	Ingredients
RubberDucky	Assembler	Beak, Body, Eyes
Body	Blowmolder	Plastic, Mold
Plastic	Store	-
Plastic	Materialiser	-
Mold	Foundry	Plastic
Mold	CNC	Wood
Beak	Symbolificator	RealBeak
RealBeak	DuckRipper2000	Duck
Eyes	Symbolificator	Spirit, Plastic
Spirit	Materialiser	-
Duck	Materialiser	-
Duck	Store	-

Exam marks will be announced on **June 28th**.